

Locality-aware Scheduling for Containers in Cloud Computing

Dongfang Zhao, Mohamed Mohamed, Heiko Ludwig

Abstract—The state-of-the-art scheduler of containerized cloud services considers load balance as the only criterion; many other important properties, including application performance, are overlooked. In the era of Big Data, however, applications evolve to be increasingly more data-intensive thus perform poorly when deployed on containerized cloud services. To that end, this paper aims to improve today's cloud service by taking application performance into account for the next-generation container schedulers. More specifically, in this work we build and analyze a new model that respects both load balance and application performance. Unlike prior studies, our model abstracts the dilemma between load balance and application performance into a unified optimization problem and then employs a statistical method to efficiently solve it. The most challenging part is that some sub-problems are extremely complex (for example, NP-hard), and heuristic algorithms have to be devised. Last but not least, we implement a system prototype of the proposed scheduling strategy for containerized cloud services. Experimental results show that our system can significantly boost application performance while preserving high load balance.

Index Terms—Cloud computing; Service computing; Containers; Data management; High-performance computing



1 INTRODUCTION

One core ingredient in cloud computing is virtualization (for example, Xen [30], Red Hat KVM [22], Microsoft Hyper-V [18], EMC/VMware vSphere [9], IBM z/VM [12]) that enables high resource utilization as well as the isolation of applications. Conventionally, the virtualization is achieved through a hypervisor that manages multiple virtual machines (VM) on the same physical machine. Nevertheless, VMs require a complete stack of software from operating system to application libraries, which takes a significant amount of time and resources before getting ready to execute the application. Recently, lightweight isolation (for example, Docker container [8]) emerges and gets well embraced because it achieves virtualization without booting a full VM; Thanks to the user-level engine, a container usually takes only a couple of seconds to start in its own logical space.

The lightweight isolation at the fine-granularity (i.e., containers), however, introduces new challenges on scheduling jobs on containers. The conventional VM-level scheduler considers VMs as standardized and independent units that are maintained by the lower-layer hypervisor. Nevertheless, containers are more proximate to each other in the sense that the controller runs at the same level (all in user space), which means the isolation between containers is not as strong as VMs. Moreover, the state-of-the-art scheduler strives to preserve load balance and is completely agnostic about the physical lay-

out of the containers, which can result in intense resource contention within and between containers (for example, local disk I/O bandwidth, network bandwidth). All of these combined together urge us to answer this research question: *how to improve the scheduler of containerized cloud services and make them more aware of their applications' data locality.*

In this paper, we make the following contributions.

- We identify a critical performance deficiency in the state-of-the-art resource scheduler in cloud computing. More specifically, we pinpoint that a round-robin policy of containerized cloud services incurs significant performance overhead from both local disk I/O and network traffic.
- We abstract the minimization of performance deficiency into optimization problems and analyze their complexities. In particular, we prove that some sub-problems are NP-hard.
- We devise heuristic algorithms for the scheduling problems. Specifically, we tackle the problem in a bottom-up fashion: we first address it from two exclusive perspectives: local disk and network traffic, respectively. Then we incorporate both aspects and develop a dynamic solution for general scenarios.
- We implement the proposed approach by extending Diego—the scheduler of the open-source project CloudFoundry [5] and IBM's Bluemix [11].
- We, experimentally, verify the correctness of the system prototype at small scales and evaluate its effectiveness at large scale. Small-scale experiments show that our system can save up to 60% network traffic with no effect to load balance. At large scales (i.e., 2,400 jobs) our system could save up to 89% network traffic with 50% skewness of load balance.

• D. Zhao is with the Department of Computer Science & Engineering, University of Nevada, Reno, NV.
Email: dzhao@unr.edu
• M. Mohamed and H. Ludwig are with the Ubiquitous Platforms Group, IBM Almaden Research Center, San Jose, CA.
Email: {mmohamed, hludwig}@us.ibm.com

The remainder of this paper first (Section 2) gives a brief background of containerized services in cloud computing and articulates the unique challenges. We formulate the problem under different scenarios, show that some of them are NP-hard, and devise and analyze heuristic algorithms in Section 3. To justify the effectiveness of the proposed algorithms and optimizations, we implement the models and algorithms in Diego (Section 4) and report experimental results (Section 5). Section 6 reviews related work on job scheduling, and Section 7 concludes this paper.

2 BACKGROUND AND MOTIVATION

As a motivating example, Figure 1 shows a typical scenario where several deployed applications depend on certain services. In the state-of-the-art scheduler of cloud computing (for example, Diego [7]) these dependencies are not considered in the decision-making process; the default strategy to deploy applications is in a straightforward round-robin fashion to achieve good load balance. As a result, the network traffic incurred by the placement decision could be huge that jeopardizes the application's performance.

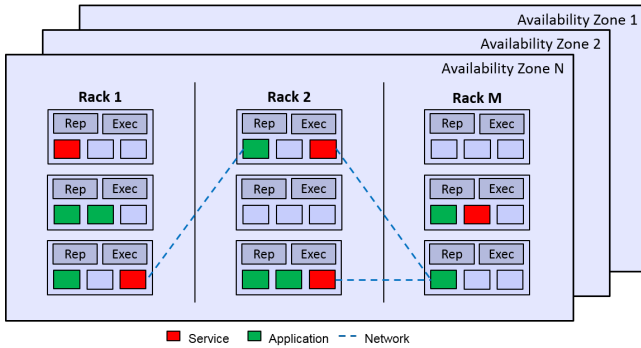


Fig. 1. Application-service affinity placement

In addition to the overhead from network traffic, another major source of performance degradation stems from the intra-node resource contention. More specifically, physical resources shared between multiple containers on the same node (either physical node or cloud instance) could be a potential bottleneck of performance. Note that, "shared resource" is not necessarily circumvented by parallelization or partitions. For instance, a conventional hard-disk drive (HDD) could serve as multiple virtual block devices for cells or dockers. The partitions of HDD meet the requirement of disjoint spaces for different applications, yet fail to isolate the resources from a performance point of view: because only a single head exists for disk seek and data access, concurrent accesses to the same HDD cause seriously degraded I/O throughput and latency.

As a concrete example for illustrating resource contention in containerized cloud service, we show three scenarios of different types of workloads deployed on the same node (or, cell): compute-intensive workloads

only, I/O-intensive workloads only, and a mix of both compute- and I/O-intensive workloads. In this oversimplified example, we allocate four containers in a VM. The compute-intensive application is a web application written in Ruby to recursively calculate Fibonacci series to number 40. The data-intensive application is a simple web portal that triggers a writing of 2 GB data to the disk.

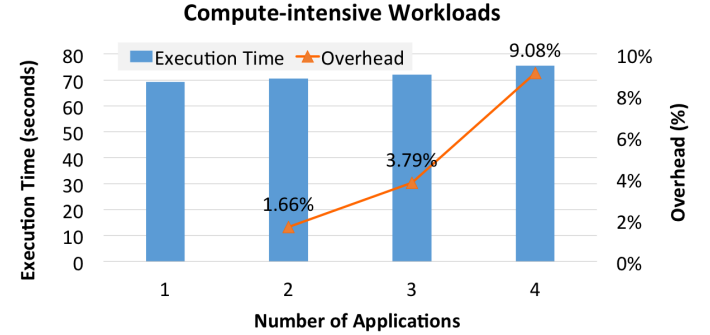


Fig. 2. Multiple compute-intensive applications on the same cell have relatively low contention

Figure 2 shows that a full-scale (i.e., 4 applications) of compute-intensive applications adds about 9% overhead comparing with the baseline. This experiment makes a strong case that the isolation of CPU resources is excellent in the context of containerized applications. After all, multiple cores within the same chip are relatively independent to each other that attribute small interaction overhead between peers.

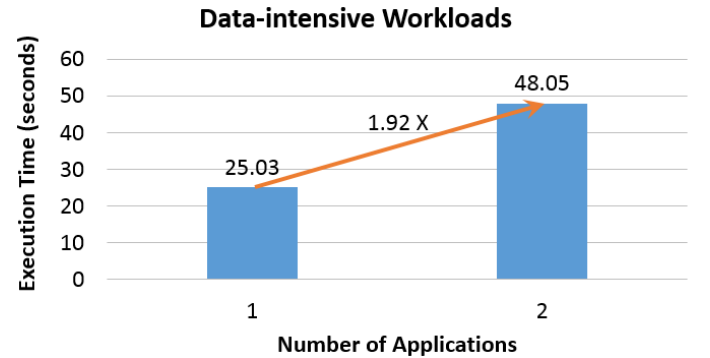


Fig. 3. Multiple data-intensive applications on the same cell cause serious resource contention and degenerated performance

Figure 3 shows that two data-intensive applications compete for the I/O bandwidth of the underlying disk bandwidth and finish in about as twice long time as the baseline. The root cause of this phenomena is that the disk I/O is not truly parallelized, although the space is exclusively partitioned for multiple containers. That is, the single head of the underlying disk is busy with the concurrent I/O requests from multiple containers; the disk becomes the performance bottleneck and a single point of failure.

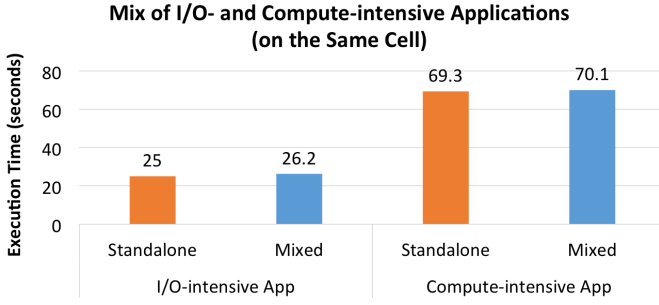


Fig. 4. Compute- and data-intensive applications deployed on the same cell do not cause noticeable contention

Lastly, Figure 4 shows that the data-intensive application and the compute-intensive application barely intervene the other when both are deployed on the same cell. In other words, the execution time of both applications are completed almost in the same time duration as their perspective baseline. Those results confirm our conjecture: the contention between distinct types of resources is light inside the same cell.

3 REDUCING RESOURCE CONTENTION WITH LOCALITY-AWARE JOB SCHEDULING

3.1 Assumption

We assume the existing mapping between service and containers do not change. That is, at least one qualified cell exists for the request. In practice, when this assumption does not hold, multiple alternatives can be applied in order to achieve the assumption; for instance, the provider might return an error message, block the request until a qualified cell becomes available, or reselect a cell that causes no contention.

A cell is the smallest isolated environment where an application can run independently. In practice, a cell can be implemented with a Docker instance. A zone is usually a data center located at the same geographic location, within which the network speed is orders of magnitude higher than that between zones.

The list of available cells is denoted by C , where C_j represents the j^{th} cell. The running applications' indexes on a particular cell are represented by $C_j.apps$. Similarly, $Z_j.apps$ indicates the list of applications running on the j^{th} zone, Z_j . Available resources of C_j can be retrieved by $C_j.cpu$, $C_j.ram$, and $C_j.disk$. The total I/O bandwidth of C_j is denoted as $C_j.io$.

The list of applications is denoted by A , where A_i represents the i^{th} application. The cell index where an application has been deployed is represented by $cell(A_i)$. The requested resources (for example, CPUs, memory, disk space) of A_i are represented by $A_i.cpu$, $A_i.ram$, and $A_i.disk$. We denote the required disk throughput of A_i as $A_i.io$. If $A_i.io$ is not explicitly specified, we assume A_i will aggressively take all the disk bandwidth of C_j .

The definitions of cells can be generalized to other entities as well. For example, if we are interested in the metrics at the zone level, then the above notations are applicable as well by replacing C to Z . Similarly, the zone where an application A_i resides is denoted by $zone(A_i)$, and the available resources of Z_j is denoted by $Z_j.cpu$, $Z_j.ram$, and $Z_j.disk$.

The dependency matrix D specifies the interaction between applications. For example, $(D_{i,j} == 1)$ means application A_i has non-trivial I/O to application A_j . Another matrix T stores the pair-wise traffic of data movement among all location units (for example, cells, physical nodes, zones). The numerical value in $T_{i,j}$ indicates the closeness between C_i and C_j . For instance, the cost of transferring data between two cells on the same physical machine is obviously lower than those on two different racks, which is then lower than those on two different locations (i.e., zones). In practice, the closeness could be network latency if users have many small I/Os, or network bandwidth if large volume of data is expected.

One important assumption of some of the following discussions is that containerized applications and services can be migrated to arbitrary locations. This is a required property for the global optimization where deployed applications need to be adjusted. However, how to migrate and how much overhead is involved with the migration is beyond the scope of this paper. In practice, both offline and online (live) migrations are available for VMs and containers (for example, Jelastic [14]).

3.2 Local Contention

The following optimization makes the number of I/O-contention nodes minimal. That is, the goal is to allocate $[A_1, \dots, A_i]$ to C (i.e., a mapping array $M[i] = j$, $1 \leq j \leq |C|$) such that:

$$\arg \min_M \sum_{j=1}^{|C|} \left(\left(\sum \{A_k.io \mid cell(A_k) == C_j\} \right) \geq C_j.io \right) \quad (1)$$

subject to

$$C_j.cpu > A_i.cpu$$

$$C_j.ram > A_i.ram$$

$$C_j.disk > A_i.disk$$

The above equation aims to minimize the number of cells where I/O-intention occurs. If the applications are not I/O-intensive, there is a high chance that no cells will have I/O intentions, in which case applications can easily fit into random cells as long as other metrics are met (e.g., CPU usage, load balance). However, the problem becomes extremely challenging when the optimal placement is not obvious: when many I/O-intensive applications are deployed to a limited number of nodes, we know there will be some nodes that are I/O throttled; the question then becomes how to minimize such bottleneck, or, minimize the number of I/O-contention nodes.

Obviously, there exists an exponential solution to this problem that is not feasible in the real-world scheduler; More efficient and practical algorithms are in need. As a matter of fact, cloud providers might not want to deploy any data-intensive application at all if no fit is available because that would cause side effect to existing applications in addition to the slowdown of the newly deployed one. A good strategy is to defer the application to either (1) any of the deployed jobs is finished, or (2) more resources (for example, new VMs, additional disks) are allocated. Then the problem becomes the following: how to allocate as many as possible applications with no outbreak of any cell's I/O bandwidth:

$$\arg \max_M \sum_{k=1}^i \left(\left(\sum \{A_h.io \mid cell(A_h) == C_j \ \&\& \ cell(A_k) == C_j\} \right) \leq C_j.io \right) \quad (2)$$

subject to

$$C_j.cpu > A_i.cpu$$

$$C_j.ram > A_i.ram$$

$$C_j.disk > A_i.disk$$

In contrast to Equation 1 that minimizes over the cells, here we try to maximize over a total number i of applications. The application A_h at the first line of Equation 2 denotes that the cell C_j has been occupied by an I/O-intensive application. Thus, we want to ensure that the subsequent application A_k at the second line would not saturate the total I/O bandwidth of the cell.

Equation 2 is a variation of the classical 0-1 *Multiple Knapsack Problem* (MKP) with two additional conditions:

- 1) all items (i.e., applications) have the same profit (we assume all applications have the same priority), and
- 2) three additional constraints (CPU, RAM, and disk space) on each item.

The MKP optimization problem itself is NP-hard, and Jansen et. al. [13] showed that the relax on profits (i.e., condition 1) does not make the problem simpler. Therefore, we propose a pseudo-heuristic algorithm with dynamic programming to meet condition 2 in addition to the optimization problem.

Algorithm 1, in essence, is a greedy algorithm sharing the same spirit of two classical algorithms: (1) the Kruskal's algorithm [15] for finding the minimal spanning tree in a graph, and (2) the best fit algorithm in memory paging allocation. Our algorithm first sorts both applications and cells according to their I/O status (Lines 1 – 2). Then, from lowest to highest, the application is assigned to the cell whose free I/O resource is larger than and closest to the application's need on the three conditions of CPU, RAM and disk space (Lines 4 – 14). Finally, if a cell is found qualified for all the requirements its I/O status is updated and its location is moved to the appropriate place in the sorted queue

(Lines 15 – 18).

Algorithm 1 Heuristically deploying I/O-intensive applications to qualified containers to reduce local I/O contention

Input: Application to be scheduled: $A_1, \dots, A_i$; Available cells C_j where $1 \leq j \leq |C|$

Output: The mapping between A and C : $M[k] := j$

- 1: Sort A_k ($1 \leq k \leq i$) in the increasing order of $A_k.io$
- 2: Sort C in the increasing order of:
 $C_j.io_avail := C_j.io - \sum \{A_m.io \mid cell(A_m) == C_j\}$
- 3: **for** $k := 1..i$ **do**
- 4: Binary search for C_j such that:
 $C_j.io_avail \geq A_k.io \ \&\& \ C_{j-1}.io_avail < A_k.io \ \text{if } j > 1$
- 5: **if** C_j does not exist **then**
- 6: **return** M
- 7: **else**
- 8: **for** ($t := j; t \leq |C|; t := t + 1$) **do**
- 9: **if** ($C_t.cpu \geq A_k.cpu \ \&\& \ C_t.ram \geq A_k.ram \ \&\& \ C_t.disk \geq A_k.disk$) **then**
- 10: $M[k] := t$
- 11: **break**
- 12: **end if**
- 13: **end for**
- 14: **end if**
- 15: **if** ($t \leq |C|$) **then**
- 16: $C_t.io_avail := C_t.io_avail - A_k.io$
- 17: Adjust the position of C_t in C (binary search)
- 18: **end if**
- 19: **end for**

Because the progress of the optimization is cached in memoization in Line 10, the time complexity of Algorithm 1 is $O(n^2)$ in total as analyzed in the following (we use n instead of i by convention to indicate the size of the input). Because presumably $i > |C|$ (otherwise the optimization problem has trivial solutions), we also use $O(n)$ to indicate the worst time when iterating on C . Both Line 1 and Line 2 take $O(n \log n)$. Line 4 takes $O(\log n)$ and Lines 8 – 13 take $O(n)$. Line 16 takes $O(1)$ and Line 17 takes another $O(\log n)$. Therefore, the most costly part is the nested loop of Line 3 and Line 8, which give us $O(n^2)$.

3.3 Network Contention

There are various granularities of network contention such as cell-to-cell, machine-to-machine, and zone-to-zone. To make the following analysis neat and clear, we analyze the zone-to-zone contention as the representative scenario. The idea and validity, however, holds true for other scenarios as well (for example, cell-to-cell, machine-to-machine). We do not differentiate an *application* and a *service* to simplify the following analysis; After all, both are just jobs from the system's point of view. Therefore, the dependency matrix D introduced in Section 3.1 is also applicable here in such a context where

($D_{i,k} == 1$) means that an application A_i is dependent on service/application A_k . The problem for an application to achieve the minimal network traffic then becomes the following. Given A_i we find Z_j ($1 \leq j \leq |Z|$) such that

$$\arg \min_j \sum_k D_{i,k} \times T_{j,zone(A_k)} \quad (3)$$

subject to

$$Z_j.cpu > A_i.cpu$$

$$Z_j.ram > A_i.ram$$

$$Z_j.disk > A_i.disk$$

It should be clear that Equation 3 only optimizes the network traffic of a single new application. This problem obviously has a polynomial solution. We can iterate all the available zones ($O(|Z|)$), and in each iteration we calculate the aggregate cost between A_i and its dependent services ($O(|A|)$). The minimal cost is also updated during each iteration. Therefore, the overall cost takes $O(|Z| \cdot |A|)$.

It should be clear that, the above discussion considers only network traffic, and load balance is out of the question. The following discussion focuses on how to tune both factors—load balance and network traffic—in a coordinated manner. That is, in the following we will have one more constraint on load balance when optimizing network traffic.

In the simplest scenario, the coefficient of variance (CV) between the loads on different zones should be under a user-defined threshold (UDT). Formally,

$$\arg \min_j \sum_k D_{i,k} \times T_{j,zone(A_k)} \quad (4)$$

subject to

$$CV(Z) \leq UDT$$

$$Z_j.cpu > A_i.cpu$$

$$Z_j.ram > A_i.ram$$

$$Z_j.disk > A_i.disk$$

Equation 4 does not add much complexity to the original criterion Equation 3. In essence, in each iteration when looping on $|Z|$ we need to recalculate CV and drop those unqualified candidates. Thus the process takes $O(|Z| \cdot (|Z| + |A|))$ in total. However, an obvious limitation exists in Equation 4: what if no qualified zone is available with the required CV?

We propose to normalize both load-balance and network-traffic criteria and apply a weight factor to each. The normalization of network traffic is measured by the ratio of the application's actual traffic divided by the aggregate bandwidth. Ideally, if all applications are executed on local nodes only, the normalized network traffic is simply zero. Formally, the normalization of network traffic (tr) for A_i on Z_j (i.e., a mapping matrix

M where $M_{i,j}$ indicates A_i is deployed on Z_j) is

$$\text{tr}(M_{i,j}) = \frac{\sum_{k=1}^{i-1} D_{i,k} \times T_{j,zone(A_k)}}{\sum_{m=1}^{|Z|} \sum_{n=1}^{|Z|} T_{m,n}} \quad (5)$$

Obviously, we have

$$\text{tr}(M_{i,j}) \in [0 \cdots 1], \forall i, j$$

which can be observed by that the total throughput of applications cannot exceed the overall bandwidth.

The normalization of load balance is defined as the adjusted coefficient of variance of the overall deployed applications on each zone. That is

$$cv(M_{i,j}) = \frac{\sqrt{(1 + |Z_j.apps| - \bar{Z})^2 + \sum_{k \neq j} (|Z_k.apps| - \bar{Z})^2}}{\sqrt{|Z|} \cdot \bar{Z}} \quad (6)$$

where

$$\bar{Z} = \frac{1 + \sum_{j=1}^{|Z|} |Z_j.apps|}{|Z|} \quad (7)$$

It should be noted that in Equation 6 we only consider the number of applications deployed to a zone as its workload. That is, we do not explicitly account resources (e.g., CPU usage, memory footprint, disk space) in our model. The reason of doing so is because the proposed model assumes all applications and zones are homogeneous for the sake of simplicity; as a result, the number of applications, implicitly, reflect the resources being used in a specific zone. Indeed, a more realistic model would have parameterized the sizes of distinct availability zones as well as the resources taken by each application, which would break the brevity of our analysis. To address that, we can assume that applications taking heterogeneous resources can be normalized into virtual applications each of which is considered identical in terms of resources. This is analogous to vCPUs [3] used in Amazon Web Services (AWS): although instance types range widely from tiny single-core machines to those high-end server-class Intel Xeon nodes, they are still comparable in terms of the number of virtual cores.

We are now ready to define the objective function with both factors. Let α and β denote the weights for the normalizations of traffic $\text{tr}()$ and coefficient of variance $cv()$, respectively. Our goal is then to find Z_j for A_i (i.e., $M_{i,j}$) such that

$$\arg \min_M F(M) \quad (8)$$

where the scoring function is

$$F(M) = \frac{\alpha \cdot \text{tr}(M) + \beta \cdot cv(M)}{\alpha + \beta}$$

subject to

$$Z_j.cpu > A_i.cpu$$

$$Z_j.ram > A_i.ram$$

$$Z_j.disk > A_i.disk$$

In Equation 8, we use the weighted arithmetic mean (AM) as the objective function. We did not choose other forms because the weights in AM have a good balance between sensitivity and complexity. We want the objective function to have a medium sensitivity: if the model does not promptly responds to an updated weight, then tuning the parameters will incur significant overhead; if the model is oversensitive, the optimal parameter can be easily missed. The cost of calculating the objective function should be minimized, as otherwise the overall cost may be significantly affected. This is why we did not introduce those more complex objective functions, such as geometric mean, harmonic mean, quadratic mean, and so on.

We in this section, without loss of generality, assume that both the traffic and the load balance contribute equally to the objective function. In practice, this can be easily changed according to specific requirements. For example, the α can be amplified by 10 times if traffic control is far more important than load balance. We will experimentally show the impact of distinct α 's and β 's in Section 5.

The complexity analysis of Equation 8 is as follows. Equation 7 can be efficiently updated in constant time assuming zones are not changed. If we maintain the difference square for each zone in memory, Equation 6 can be updated also in constant time. Nevertheless, it takes $O(|Z|)$ to update Equation 6. In Equation 5, every dependency is checked taking $O(|A|)$. Therefore, the overall time is $O(|Z| \cdot (|Z| + |A|))$. That is, although adding both network traffic and load balance into consideration looks more complicated than the simple threshold-check approach (i.e., Equation 4), the former one essentially takes as much time in the same order of magnitude as the latter.

The problem becomes significantly harder (even without considering load balance) with a subtle change when we consider scheduling a batch of applications or all the applications globally. In essence, instead of the linear problem on a single dimension of $|Z|$, we are now facing a combinatorial problem of placing $|A|$ applications on $|Z|$ zones such that:

$$\arg \min_M \sum_i \sum_k D_{i,k} \times T_{j,zone(A_k)} \quad (9)$$

subject to

$$Z_j.cpu > A_i.cpu$$

$$Z_j.ram > A_i.ram$$

$$Z_j.disk > A_i.disk$$

A brute-force solution would obviously take exponen-

tial time, $O(|Z|^{|A|})$. We will prove the problem of Equation 9 is NP-hard by reducing it to the *Traveling Salesman Problem* (TSP). The zone matrix can be represented as a graph whose vertexes represent the zones and edges represent the network traffic between two zones, which is a similar setup of TSP. Instead of TSP where we try to find the shortest path covering all vertexes, we are now trying to find the smallest summation of edges arbitrarily determined by the dependency matrix. If we specify the dependency matrix where each application only depends on two neighbor applications and enforce one zone holding only one application, then our global-network-minimization problem is reduced to TSP.

This paper is focused on the optimization problem with regard to a single application. Therefore, the algorithm, analysis, and evaluation are all concentrated on how to minimize the contention when a new application is deployed. We leave the optimization of all applications as an open question to the community as well as one of our future works.

3.4 Generalized Model

Both types of contention—local (Section 3.2) and network (Section 3.3)—we have discussed so far are disconnected. A strategy from the entire system's point of view is the focus of this section. As a case in point, when a new application is to be deployed, should it be scheduled to avoid local or network contention? More specifically, although we should try to assign an application to the same cell or zone of its dependent service, is it wise to do so if both the application and its dependent service incur significant disk I/Os? Previous sections partially answer the above question from two exclusively aspects; The following will discuss how to coordinate both in a systematic manner.

The idea is to still maintain the network traffic as little as possible with an additional condition that no local disk bandwidth is deplete. Therefore, Equation 8 is slightly changed into

$$\arg \min_M F(M) \quad (10)$$

where

$$F(M) = \frac{\alpha \cdot tr(M) + \beta \cdot cv(M)}{\alpha + \beta}$$

subject to

$$Z_j.cpu > A_j.cpu$$

$$Z_j.ram > A_j.ram$$

$$Z_j.disk > A_j.disk$$

$$Z_j.io > A_j.io$$

The newly added condition on $Z_j.io$ and $A_j.io$ indicates that (1) the system maintains a zone-level metadata for its I/O bandwidth usage, and (2) the I/O requirement of applications is known upfront. While the first one is straightforward to implement, the second one can be

implemented as a probe service or as a machine learning algorithm based on historical usage.

3.5 Broader Applicability and Limitations

Containers, as a lightweight alternative to VMs, by nature become a good candidate for the new computing paradigm where the workload (computation, storage, communication) is pushed to the edge—edge computing or fog computing. While a concrete definition of fog computing is still under active debate [32], we strongly believe pushing workloads toward end devices would become a trend with the enabling technology of containers in many computing communities including scientific computing, social network, web services, and so forth.

Although this paper is concentrated on the differentiated performances across multiple availability zones in the conventional geographically-distributed clouds, the idea and approach of minimizing local- and network-contention can be applied to more computing paradigms including fog computing. Essentially, in fog computing the computing and storage units are slimmer and more distributed than data centers—they are everywhere from personal electronic device (for example, smart phones, smart watch) to enterprise facilities such as weather sensors and surveillance monitors. Despite a radical change from provider-hosted resources to ubiquitous dispersion, one of the key challenges remains: the computation should be deployed as close as possible to its data to avoid the costly network congestion.

The locality problem is more challenging in the fog computing from the following two perspectives.

- The network is more heterogeneous and volatile than cloud computing. In fog computing, the network could be mobile network, WiFi signal, Ethernet connection, Bluetooth, and so forth, which is completely another story comparing with the relatively stable Internet connection in cloud computing. As a case in point, a mobile phone's latency is significantly more volatile than that of a physical node in the data center. Consequently, the matrix we introduce to characterize the network traffic needs to be frequently updated.
- The scale is significantly larger. In cloud computing, when discussing availability zones, they might be in orders of $O(10)$. The entity granularity is coarse: the machines within the same data center are considered as homogeneous nodes so the entire data center might be treated as a single entity. In fog computing, however, each device is a potential entity because there is little association between devices. As a result, the traffic matrix would be proportional to the total number of devices instead of $O(10)$. Note that there could be millions of users or more that could invalidate the matrix approach we have discussed so far in this paper.

4 IMPLEMENTATION

We have implemented the hybrid model discussed in Section 3. We modified the Diego [7] source code to allow users to choose whether locality should be considered in the scheduling decision. In particular, a new module called `locality.go` is developed among other changes to the existing modules such as `auctionrunner.go` and `scheduler.go`.

The `Locality` class¹ is defined in Figure 5. The `dep` and `traffic` fields represent the dependency and traffic matrices discussed in Section 3. The `appZone` field is a hashmap from an existing application to its assigned zone. The `zones` field is a sortable array of all the available zones. The `zoneAppNum` field records the number of running applications on a particular zone.

```
type Locality struct {
    dep map[string]map[string]uint
    traffic map[string]map[string]float64
    appZone map[string]string
    zones sort.StringSlice
    zoneAppNum map[string]int
}
```

Fig. 5. Definition of the `Locality` class

A `Locality` reference is instantiated and initialized in the main scheduling entity named `auctionrunner`. That is, only one instance of `Locality` (for example, the dependency matrix, the traffic matrix, and so forth) is maintained. The `Locality` reference is then passed to the `scheduler` object that could call either the vanilla placement algorithm or the method named `AssignCellWithMinTraffic(app string)` exposed by `Locality`.

A simplified diagram of key components is shown in Figure 6. Two major packages are involved: `auctionrunner` and `models`.

- The `auctionrunner` package is responsible for all the auction-related scheduling, where most changes were made to the vanilla Diego. It takes the deployment request and initializes a `Locality` object (step ①), whose reference is then passed to the `Scheduler` class (step ②). The scheduler deploys the application to the cell according to the placement algorithm specified in the `Locality` class.
- The `models` package maintains all the abstraction models in Diego, which was extended in the `DesiredLRP` to also manage the application dependencies. That is, the `Locality` class retrieves the application dependency from the `DesiredLRP` (step ③) that is originally specified in the manifest file by the users.

1. Technically, it is a `struct` in the Go programming language.

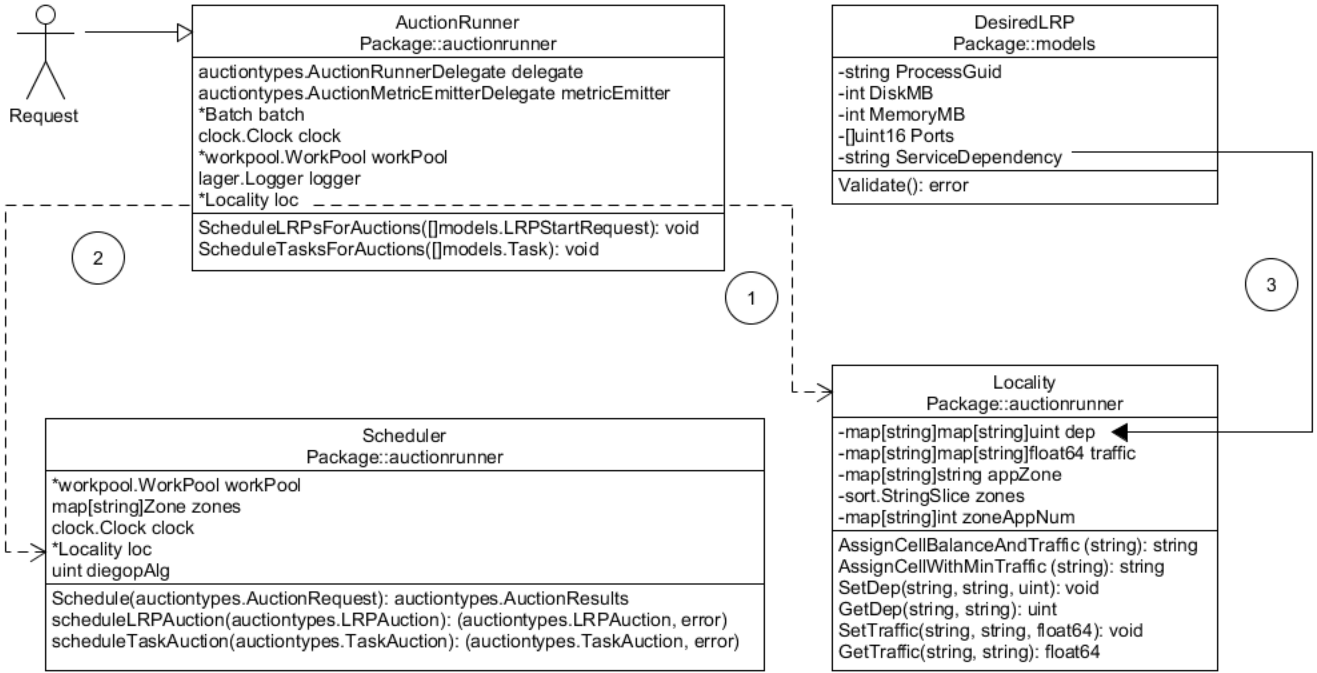


Fig. 6. Diego [7] extended with the `Locality` class

The dependency matrix between applications is specified in the manifest file when the application is launched. The information is saved in a new field named `ServiceDependency` in the `DesiredLRP` class. Later on, when scheduling the application the dependency information is pulled out for the dependency matrix.

5 EVALUATION

We verified the correctness of the proposed scheduling algorithms at small scales (i.e., eight applications), then evaluated its effectiveness at large scales (i.e., 2,400 applications/containers), both by leveraging the built-in simulation module in Diego. The experiments were carried out on 64-bit Ubuntu operating systems with quad-core CPUs and 64 GB memory. The original Diego scheduler was released as v0.13 and was primarily implemented with the Go programming language v1.5; each docker image was installed with the `cflinuxfs2` stack supported by Cloud Foundry. Note that, we assume the containers in this discussion are of the same capacity for clear presentation; in the real world the containers might have different captivities and yet they can be normalized to the same and the following evaluation will be still valid. The simulation module arbitrarily creates various types of workloads (i.e., with different number of applications, containers, cells, zones, etc.) that triggers the Diego scheduler. While the workloads are simulated, the scheduler itself is indeed executed (not simulated).

We started with an oversimplified example: two cells (REP-1 and REP-2) provisioned in two zones (Z0 and

Z1), respectively. Each of the cells has a capacity of eight containers. Eight applications are to be deployed, namely A0 to A7. The vanilla placement algorithm in Diego, unsurprisingly, assigns four applications on each zone for the sake of load balance as the simulator output shows in Figure 7 where “+” indicates a deployed application and “.” represents a free container.

```
Distribution
[Z0] REP-1: ++++....
[Z1] REP-2: ++++....
```

Fig. 7. Distribution of eight applications on two zones with Diego

Now let us consider the same set of applications are being deployed with the proposed locality-aware algorithms. Applications A0 to A7 are deployed to our 2-zone cloud in a serial manner where each application is dependent on other applications specified as 1 in Figure 8. For instance, A0 and A1 do not have any dependent applications (or, services), while A2 is dependent on A1 and A3 is dependent on A0. For $i := 4..7$, $A(i)$ is dependent on $A(i-1)$.

The traffic matrix between zones is shown in Figure 9. Here we assume the network traffic across different zones is symmetric, which is why the matrix is also symmetric. The unit of the numbers does not matter as long as they accurately reflect the relative traffic. In practice, “traffic” could be end-to-end latency for small file accesses, network throughput for data-intensive workloads, or a mix of forth. Due to the huge performance difference between local and remote traffic,

Dependency	A0	A1	A2	A3	A4	A5	A6	A7
A0	0	0	0	0	0	0	0	0
A1	0	0	0	0	0	0	0	0
A2	0	1	0	0	0	0	0	0
A3	1	0	0	0	0	0	0	0
A4	0	0	0	1	0	0	0	0
A5	0	0	0	0	1	0	0	0
A6	0	0	0	0	0	1	0	0
A7	0	0	0	0	0	0	1	0

Fig. 8. Dependency matrix of applications

Traffic	Z0	Z1
Z0	10	100
Z1	100	10

Fig. 9. Traffic matrix of zones

we should try to assign dependent applications on the same zone if possible from performance's perspective. If either zone yields the same traffic, then we pick the one with fewer running applications (from the `zoneAppNum` field in Figure 5) for the sake of load balance. Since we assume all containers are of the same capacity, the above decisioning is equivalent to choosing the zone with the most capacities when multiple zones qualify for the application.

In this example, the traffic across two zones is 10 times larger than the local one. We chose 10 by observing that the inter-zone latency is roughly 10 times larger than intra-zone on Amazon EC2: the latency within North California is 34 milliseconds while the latency between North California and Singapore 385 milliseconds [6]. In the following discussion, we use 10 and 100 for clear presentation instead of the raw measurements (i.e., 34 and 385).

We are now ready to understand why our proposed scheduler places six applications in Z0 and two others to Z1, as shown in Figure 10. Since neither A0 nor A1 has any dependency, A0 is assigned to Z0 and A1 is assigned to Z1. For A2, because it is dependent on A1 it is deployed to Z1 as well. Yet, because A3 is dependent on A0, it is deployed to Z0. Similarly, A4 ... A7 are deployed to Z0 because their dependencies are all on Z0. Therefore, we observe six applications are on Z0: A0, A3, ..., A7; two applications are on Z1: A1 and A2.

```

Distribution
[Z0] REP-1: ++++++..
[Z1] REP-2: ++.....

```

Fig. 10. Distribution of the same eight applications on two zones with Diego/P

The overall application traffic is calculated as follows. Recall that according to the strict load-balance rule of Diego, all even-numbered applications are deployed to

Z0 and all odd-numbered applications are deployed to Z1. Unfortunately, all of A2, ..., A7 have dependencies (refer to Figure 8) residing on the other zone of its own. Therefore, the overall traffic is $100 \times 6 = 600$. On the other hand, in our proposed scheduler all the six applications have their dependencies in the same zone, resulting in the overall traffic $6 \times 10 = 60$. That is, with locality-aware scheduling we are able to migrate costly inter-zone traffic into lighter intra-zone traffic.

While the network traffic is greatly reduced with the proposed scheduling strategy, one obvious limitation lies on the skewness of job placement: Z0 has 6 applications: 300% of Z1. Statistically, it results in significant variance: $\frac{(6-4)^2 + (2-4)^2}{2} = 4$. That is, the coefficient of variation (CV) is $\sqrt{4}/4 = 50\%$. An extremely low network traffic is definitely desirable in terms of performance; yet it indicates a low network utilization from vendor's perspective. Therefore, we are interested in trading off a portion of network traffic to (significantly) improve CV.

We first re-ran the same workloads as discussed above by assigning the same weight on load balance and locality. That is, $\alpha = 1$ and $\beta = 1$. As shown in Table 1 (first two rows), the allocation is exactly the same as when considering no load-balance at all (see Figure 10). This can be best explained by that the coefficient of variance between zones is not significant enough to impact the weighted function that is dominated by the network traffic.

TABLE 1

Function scores and placement of eight applications on two zones (Z0, Z1) in two setups: $\{F^1: \alpha = 1, \beta = 1\}$ and $\{F^2: \alpha = 1, \beta = 2\}$ (selected zone is underlined)

	App	A0	A1	A2	A3	A4	A5	A6	A7
Z0	F^1	<u>.50</u>	.50	.39	<u>.02</u>	<u>.12</u>	<u>.19</u>	<u>.24</u>	<u>.27</u>
Z1	F^1	.50	<u>.00</u>	<u>.19</u>	.02	.12	.19	.24	.27
Z0	F^2	<u>.67</u>	<u>.67</u>	<u>.37</u>	<u>.02</u>	<u>.15</u>	<u>.24</u>	<u>.25</u>	<u>.15</u>
Z1	F^2	.67	<u>.00</u>	<u>.24</u>	.02	.15	<u>.15</u>	<u>.11</u>	.15

Then we increase the weight on load balance (i.e., ν_C): $\alpha = 1$ and $\beta = 2$. The results are reported in the bottom two rows in Table 1. Now we observe A5 and A6 are placed in Z1 rather than Z0. That is, the increased weight on load balance counterfeits the network traffic on these two applications. Also note that, even though in this scenario both Z0 and Z1 have the same number of applications just like in vanilla Diego (Figure 7), the applications are different in each zone.

In the scenario of $\{\alpha = 1, \beta = 2\}$, the overall network traffic is 240 because $A0 = A1 = 0$, $A2 = A3 = A4 = A6 = 10$, and $A5 = A7 = 100$. While the traffic usage is higher than the strict network-optimal strategy, the ratio is still relatively low: $240/(220 \times 8) = 13.6\%$. In return, the system is perfectly load-balanced. Comparing with the vanilla Diego scheduler, our proposed approach achieves the same load balance but reduce the traffic from 600 to 240 — saving $1 - \frac{240}{600} = 60\%$ network bandwidth.

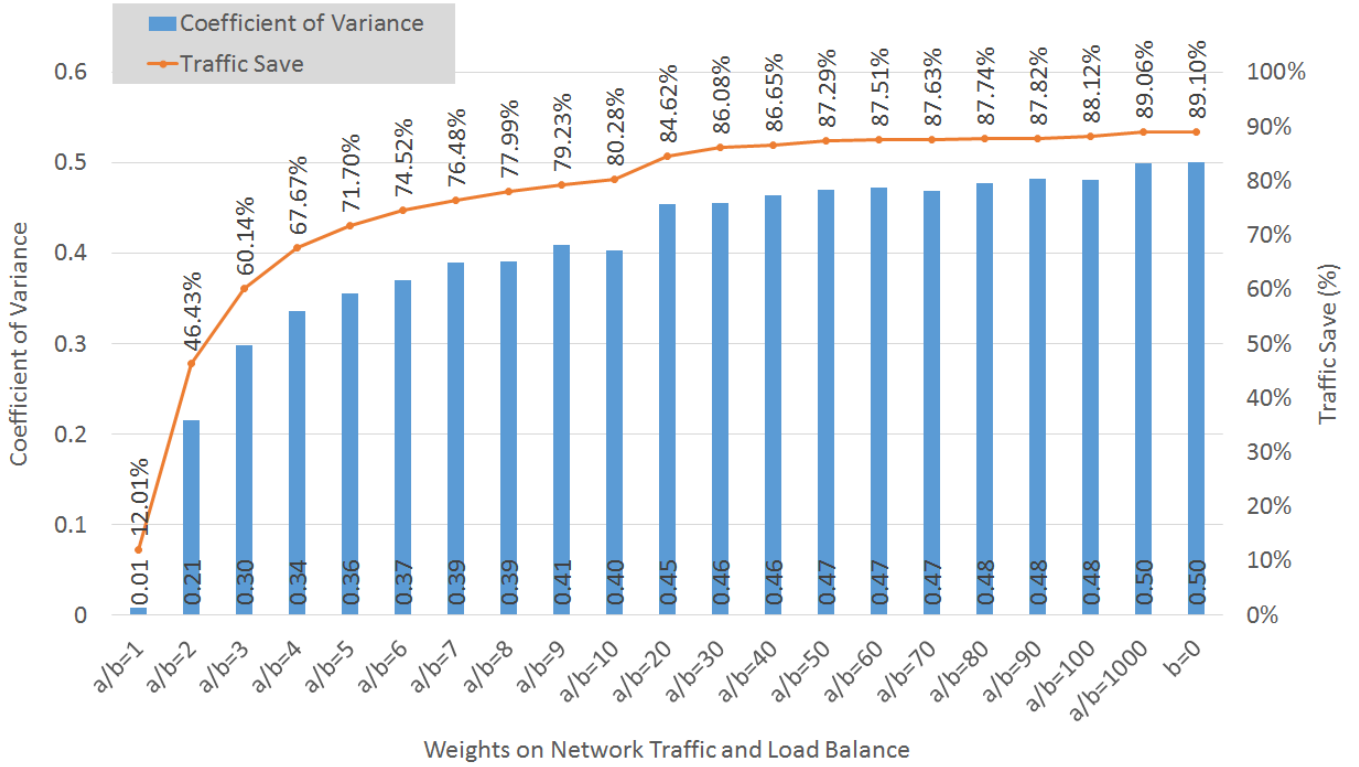


Fig. 11. Network traffic and load balance of 2,400 jobs on 30 zones with different configurations

We scale the experiment out to 2,400 applications and 30 zones. The traffic saving and load balance is reported in Figure 11 on various combinations of parameters. Several interesting observations are discussed in the following.

First, the right-most column reports that the upper-bound of performance improvement is 89% where load-balance is not considered at all (i.e., $\beta = 0$). This result is understandable because the inter-zone network is $10\times$ costly than intra-zone, which means diverting most inter-node traffic to within the zone roughly results in about 10% network traffic.

Second, the effect of locality weight at smaller scales (i.e., $\frac{\alpha}{\beta} \leq 5$) is more significant to both traffic save and load balance. When the ratio increases from 1 to 5, the traffic save increases 60% (from 12% to 72%) and the coefficient of variance increases 0.35 (from 0.01 to 0.36). On the other hand, when the ratio increases from 5 to 1,000, the traffic save increases only 17% (from 72% to 89%) and the coefficient of variance increases only 0.14 (from 0.36 to 0.50). Therefore, a good practice to choose α and β is to only sweep on the parameter space at small scales of the ratio between α and β .

We visually demonstrate how the proposed locality-aware scheduler places the applications to different containers and zones under different criteria. We first show two representative scenarios in Figure 12 and Figure 13, respectively. Both experiments are deployed to the largest scale, i.e., 2,400 jobs on 30 zones, with different parameters. Afterwards, we will illustrate two

extreme cases (Figure) where α is set to 1 and 1,000, respectively.

The first setup in Figure 12 shows how our proposed scheduler, when $\alpha = 2$ and $\beta = 1$, makes a better trade-off between data locality and load balance compared to the case of $\alpha = 1$ and $\beta = 1$ where only 12% network traffic is reduced although it is perfectly load balanced. Specifically, this placement slightly gives up load balance by deploying more workloads in the middle containers (i.e., coefficient of variance 0.21 per Figure 11). In return, it helps save almost half of the network traffic (i.e., 46.43% per Figure 11).

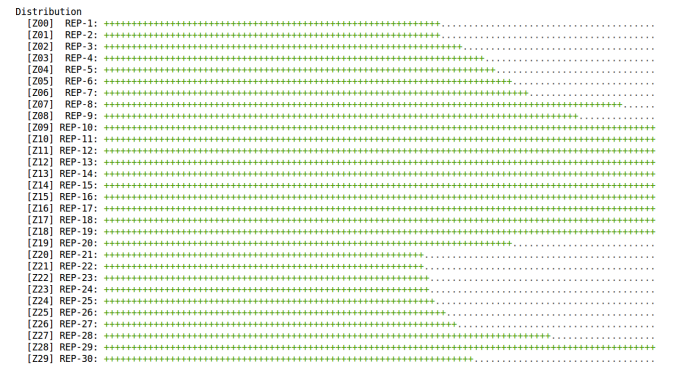


Fig. 12. Placement of 2,400 jobs on 30 zones when $\alpha = 2$, $\beta = 1$

The second setup in Figure 13, where $\alpha = 100$ and $\beta = 1$, represents a placement decision on the other

micro-architectural resources (disk I/O bandwidth and network traffic) at a finer granularity (i.e., containers).

Scheduling is also researched to improve other aspects of the system. For example, one of the most interesting angles is power consumption and electricity bill [10, 17, 31]. Those works showed that a well-tuned job scheduler could significantly reduce the power consumption and electricity bill. This work can be combined with the aforementioned to further reduce the financial investment if network traffic is considered as a major portion of the operation cost.

Although Docker [8] is one of the most popular implementation for container services, it per se does not expose any data locality for the deployed containers. While Kubernetes [16] does provide management of container services in the form of “pods”, what it does is to assign containers to the pod without much optimization inspired by the application’s I/O patterns. In contrast, this work proposes a series of models and algorithms to allow users to specify a degree of skewness of load balance in order to achieve better data locality and consequently the performance from a global perspective of the entire cluster.

There are also important works in scheduling for better load balance from distributed systems and high-performance computing (HPC) communities, such as [1, 28, 33]. In particular, Wang et al. [27] proposed an ephemeral burst buffer file system, a breakthrough to alleviate the conventional I/O pressure that has existed in HPC systems for decades. We believe the ephemeral burst buffer can be leveraged by the community of cloud computing as well, as it is orthogonal to the algorithms and models proposed by this paper. Thus, applying the ephemeral burst buffer file system proposed in [27] will likely further improve the performance of many cloud computing applications.

7 CONCLUSION AND FUTURE WORK

The emerging containerized cloud services, although provide a finer granularity of visualization, as any other new technology inevitably poses new challenges. Among them, the performance of applications and services is of great interests not only from users’ perspectives but also the more performance-aware service-level agreement between the providers and customers. This paper identifies the containerized service scheduler as one of the performance deficiencies in the state-of-the-art design of PaaS. More specifically, we pinpoint that a simple round-robin policy of placing applications/services on containers incurs significant performance overhead from both local disk I/O and network traffic.

The first step we take to tackle the challenge is formulating it into optimization problems and analyze their complexities. Unsurprisingly, some of the problems turn out to be NP-hard. To that end, we devise heuristic algorithms and solutions for different scenarios. The proposed technique is implemented and evaluated in the

de facto PaaS platform—CloudFoundry [5]. We justify the correctness of the proposed work at small scales, which shows that the network traffic could be reduced by 60% with no side effect to load balance. In order to demonstrate its effectiveness at large scale, we carry out large-scale experiments (i.e., 2,400 applications) showing that the proposed scheduling strategy is able to save up to 89% network traffic by trading off 50% of load balance.

There are three major future research directions with this work. First, an application’s I/O requirement would be better quantified. The current design assumes that the user would specify this property in a configuration file (e.g., manifest). An automatic and systematic approach is desirable to make the system more accurate that could also reduce human intervening. Second, an efficient approach to achieve the global optimization of network traffic is in need. This paper scrapes the surface of this problem—proving it NP-hard—and leaves developing efficient algorithms to the optimization solution as an open question to future work. Third, the models, algorithms, and prototypes discussed in this work will be extended to a larger context and integrated to an orchestrated cloud service. Currently, this work assumes that the storage layer is a shared-nothing cluster of local Linux file systems (e.g., ext4); we will be working on providing a GPFS [24] interface so that other communities (e.g., scientific data management, high performance computing) would also benefit from the scheduling techniques presented in this paper.

ACKNOWLEDGMENT

This work is supported in part by the Big Data Research Initiatives sponsored by the Nevada System of Higher Education (on behalf of the University of Nevada, Reno), the AWS Research Grants from Amazon, and the Azure Research Award from Microsoft. Some preliminary work was conducted when D. Zhao worked at IBM Almaden Research Center. The authors would like to thank Prof. Ioan Raicu (Illinois Institute of Technology), Prof. Ion Stoica (University of California, Berkeley), and Prof. Magdalena Balazinska (University of Washington) for insightful discussions.

REFERENCES

- [1] F. Ahmad, S. T. Chakradhar, A. Raghunathan, and T. N. Vijaykumar, “Shufflewatcher: Shuffle-aware scheduling in multi-tenant mapreduce clusters,” in *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, 2014, pp. 1–12.
- [2] J. Ahn, C. Kim, J. Han, Y.-R. Choi, and J. Huh, “Dynamic virtual machine scheduling in clouds for architectural shared resources,” in *Proceedings of the 4th USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*, 2012.
- [3] AWS vCPU, “<https://aws.amazon.com/ec2/virtualcores/>,” Accessed December 3, 2017.

- [4] Y. Cheng, W. Chen, X. Chen, B. Xu, and S. Zhang, "A user-level numa-aware scheduler for optimizing virtual machine performance," in *Advanced Parallel Processing Technologies*, 2013, vol. 8299.
- [5] Cloud Foundry, "http://docs.cloudfoundry.org/", Accessed July 17, 2015.
- [6] CloudWatch, "http://www.cloudwatch.in/", Accessed July 15, 2015.
- [7] Diego Project, "https://github.com/cloudfoundry-incubator/diego-release," Accessed July 16, 2015.
- [8] Docker container, "https://github.com/docker/docker," Accessed July 16, 2015.
- [9] EMC/VMWare vSphere, "https://www.vmware.com/products/vsphere/", Accessed July 17, 2015.
- [10] J. Hikita, A. Hirano, and H. Nakashima, "Saving 200kw and \$200 k/year by power-aware job/machine scheduling," in *IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*, 2008.
- [11] IBM Bluemix, "http://www.ibm.com/cloud-computing/bluemix/", Accessed July 17, 2015.
- [12] IBM z/VM, "http://www.vm.ibm.com/overview/", Accessed July 17, 2015.
- [13] K. Jansen, F. Land, and K. Land, "Bounding the running time of algorithms for scheduling and packing problems," in *Proceedings of the 13th International Conference on Algorithms and Data Structures (WADS)*, 2013.
- [14] Jelastic, "http://ops-docs.jelastic.com/cluster-features#c," Accessed July 16, 2015.
- [15] J. Kruskal, Joseph B., "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, 1956.
- [16] Kubernetes, "https://github.com/kubernetes/kubernetes," Accessed July 24, 2017.
- [17] O. Mammela, M. Majanen, R. Basmadjian, H. De Meer, A. Giesler, and W. Homberg, "Energy-aware job scheduler for high-performance computing," *Computer Science - Research and Development*, vol. 27, no. 4, 2012.
- [18] Microsoft Hyper-V, "https://technet.microsoft.com/library/hh831531.aspx," Accessed July 17, 2015.
- [19] Mira, "http://www.alcf.anl.gov/user-guides/mira-cetus-vesta," Accessed July 17, 2015.
- [20] F. Paraiso, S. Challita, Y. Al-Dhuraibi, and P. Merle, "Model-driven management of docker containers," in *2016 IEEE 9th International Conference on Cloud Computing (CLOUD)*, 2016.
- [21] J. Rao, K. Wang, X. Zhou, and C.-Z. Xu, "Optimizing virtual machine scheduling in numa multicore systems," in *Proceedings of the 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, 2013.
- [22] Red Hat KVM, "http://www.redhat.com/en/files/resources/en-rh-kvm-kernal-based-virtual-machine.pdf," Accessed July 17, 2015.
- [23] S. G. Saez, V. Andrikopoulos, R. J. Sanchez, F. Leymann, and J. Wettinger, "Dynamic tailoring and cloud-based deployment of containerized service middleware," in *2015 IEEE 8th International Conference on Cloud Computing (CLOUD)*, 2015.
- [24] F. Schmuck and R. Haskin, "GPFS: A shared-disk file system for large computing clusters," in *Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*, 2002.
- [25] Top500, "http://www.top500.org/list/2014/06/", Published June 2014; Accessed September 5, 2014.
- [26] K. Wang, X. Zhou, T. Li, D. Zhao, M. Lang, and I. Raicu, "Optimizing load balancing and data-locality with data-aware scheduling," in *Proceedings of IEEE International Conference on Big Data (IEEE BigData)*, 2014.
- [27] T. Wang, K. Mohror, A. Moody, K. Sato, and W. Yu, "An ephemeral burst-buffer file system for scientific applications," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 69:1–69:12.
- [28] Y. Wang, R. Goldstone, W. Yu, and T. Wang, "Characterization and optimization of memory-resident mapreduce on hpc systems," in *2014 IEEE 28th International Parallel and Distributed Processing Symposium*, 2014, pp. 799–808.
- [29] C. Weng, Q. Liu, L. Yu, and M. Li, "Dynamic adaptive scheduling for virtual machines," in *Proceedings of the 20th International Symposium on High Performance Distributed Computing (HPDC)*, 2011.
- [30] Xen, "http://www.xenproject.org/", Accessed July 17, 2015.
- [31] X. Yang, Z. Zhou, S. Wallace, Z. Lan, W. Tang, S. Coghlán, and M. E. Papka, "Integrating dynamic pricing of electricity into energy aware scheduling for hpc systems," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2013.
- [32] S. Yi, C. Li, and Q. Li, "A survey of fog computing: Concepts, applications and issues," in *Proceedings of the 2015 Workshop on Mobile Big Data*, 2015.
- [33] M. Zaharia, D. Borthakur, J. Sen Sarma, K. Elmelegy, S. Shenker, and I. Stoica, "Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling," in *Proceedings of the 5th European Conference on Computer Systems*, 2010, pp. 265–278.
- [34] Y. Zhai, L. Yin, J. Chase, T. Ristenpart, and M. Swift, "Cqstr: Securing cross-tenant applications with cloud containers," in *Proceedings of the Seventh ACM Symposium on Cloud Computing (SoCC)*, 2016.
- [35] D. Zhao, "Toward real-time and fine-grained monitoring of software-defined networking in the cloud," in *2016 IEEE 9th International Conference on Cloud Computing (CLOUD)*, 2016.
- [36] Z. Zhou, X. Yang, D. Zhao, P. Rich, W. Tang,

J. Wang, and Z. Lan, "I/o-aware batch scheduling for petascale computing systems," in *Cluster Computing, IEEE International Conference on*, 2015.



Dongfang Zhao is an Assistant Professor in the Department of Computer Science & Engineering at University of Nevada, Reno. His research interests span data management systems, high-performance computing, cloud computing, distributed systems, and machine intelligence. He obtained a PhD in computer science from Illinois Institute of Technology, Chicago, and completed his postdoctoral fellowship in the School of Computer Science & Engineering at University of Washington, Seattle.



Mohamed Mohamed is a Research Staff Member in the Ubiquitous Platforms Group, IBM Almaden Research Center, San Jose, California. He is working on different projects that are primarily related to PaaS including data persistence management and SLAs management. He is one of the main contributors in designing and implementing rSLA language and framework, as well as CloudFoundry's persistence support. He holds a PhD in computer science from Telecom SudParis, Institut Mines-Telecom, Paris, France.



Heiko Ludwig is a Research Staff Member and Manager with IBM Almaden Research Center in San Jose, California. Leading the Ubiquitous Platforms research group, he is currently working on topics related to container-based systems and Platform as a Service (PaaS), IOT platforms, SLA and quality management, as well as platform security. He holds a Master's degree and a PhD in information systems from Otto-Friedrich University Bamberg, Germany.